

PaiseWise — Phase 3

Task 1: Conversational Assistant Technical Design Document

Version 1.0 | Technical Design for Team Review

Item	Design
API	FastAPI — POST /ai/chat
Session state	Redis — chat:{userId}:{sessionId}, TTL 4 hours
Persistent storage	PostgreSQL
Context	Last 10 messages + user profile + portfolio summary
Safety	Topic classification + deterministic guardrail + response validation
AI layer	RAG + replaceable LLM provider
Streaming	FastAPI StreamingResponse
Feedback	POST /ai/chat/feedback; thumbs up/down

1. Executive Summary

- This document defines the technical design for the PaiseWise conversational financial-learning assistant required in Phase 3 Task 1.
- The design focuses on a modular, secure and scalable API architecture around FastAPI, with Redis for short-lived conversation state, PostgreSQL for durable application data, a replaceable RAG/LLM layer, explicit guardrails, streaming responses and response feedback.
- The design deliberately keeps Task 1 focused on architecture and technical foundation. RAG implementation is specified as the next implementation stage, while the performance target and feedback analytics are expanded in Task 3.

2. Source Requirements

- Design POST /ai/chat with message history.
- Use a conversation context window containing the last 10 messages, user profile and portfolio summary.
- Plan guardrails and identify question types that should be deflected to a SEBI advisor.
- Store chat history in Redis per session with a 4-hour TTL.
- Research and design FastAPI streaming for a typewriter effect in the mobile app.
- Design thumbs-up/down feedback for each AI response and store feedback in the database.
- Produce this technical design and obtain team review.

3. Design Goals

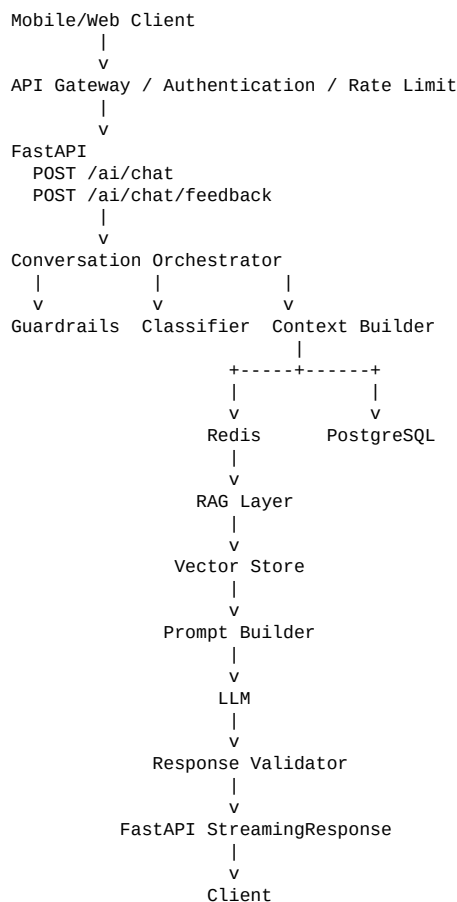
- Safety first: the assistant is a financial-learning assistant and must not provide specific investment advice.
- Bounded context: use only the required recent conversation context rather than sending unlimited history.
- Low latency: Redis is used for frequently accessed session state and streaming is used for perceived responsiveness.
- Maintainability: keep API, orchestration, guardrails, retrieval, model and persistence responsibilities separated.
- Provider flexibility: hide the LLM and embedding implementations behind service interfaces so the model/provider can change without redesigning the API.
- Testability: every critical decision point should be unit-testable without requiring a live LLM.

4. Recommended Technology Stack

- Backend: Python + FastAPI + Uvicorn.

- Validation: Pydantic.
- Session state/cache: Redis.
- Persistent database: PostgreSQL.
- Database access: SQLAlchemy.
- RAG: embedding model + vector database; exact provider can be selected during Task 2 based on infrastructure constraints.
- LLM: configurable provider behind an LLM service interface.
- Testing: Pytest.
- API verification: REST Client / Swagger UI.
- Configuration and secrets: environment variables and .env for local development.

5. High-Level Architecture



- The API layer should be stateless. Redis owns temporary session state, PostgreSQL owns durable records, and the AI layer remains replaceable.

6. Component Responsibilities

- FastAPI API layer: request validation, authentication integration, routing and streaming.
- Conversation Orchestrator: coordinates classification, guardrail decisions, context loading, generation, validation and persistence.
- Topic Classifier: assigns jargon, product, market or personal advice categories.
- Guardrail Service: blocks the normal generation path for personal-advice requests and returns the required safe deflection.
- Context Builder: assembles system instructions, user profile, portfolio summary, last 10 messages and later retrieved RAG context.
- Redis Repository: stores session messages under chat:{userId}:{sessionId} and applies the 4-hour TTL.

- RAG Service: embeds the question, retrieves relevant knowledge and supplies grounded context; this is implemented in Task 2.
- LLM Service: exposes a provider-neutral generate/stream interface.
- Response Validator: checks generated text for prohibited content before delivery; this is expanded in Task 3.
- Feedback Repository: stores response feedback in PostgreSQL.

7. Chat Request Lifecycle

1. Client sends POST /ai/chat.
 2. Validate userId, sessionId and message.
 3. Authenticate/authorize the user.
 4. Classify the question.
 5. Apply guardrails.
 6. If personal advice, return the required deflection and stop.
 7. Load the session history from Redis.
 8. Load the user's profile and portfolio summary.
 9. Build the bounded conversation context.
 10. Retrieve relevant knowledge through the RAG layer.
 11. Build the grounded system/user prompt.
 12. Stream the LLM response through FastAPI StreamingResponse.
 13. Validate the generated response before it is delivered.
 14. Persist the resulting conversation state and response metadata.
- Implementation should preserve the distinction between deterministic safety decisions and probabilistic LLM generation.

8. Conversation Context Design

- The required context window is: last 10 messages + user profile + portfolio summary.
- Recommended prompt/context order: system policy → user profile → portfolio summary → recent conversation → retrieved knowledge → current question.
- Do not send unlimited history. A bounded window makes token usage and latency predictable.
- User profile fields specified by the later task include goal, level and KYC status; holding summary is also part of the required profile/portfolio context.

9. Redis Session Design

- Key format: chat:{userId}:{sessionId}.
- TTL: 4 hours = 14,400 seconds.
- Stored value: ordered role/content messages.
- On each request: GET history → append user/assistant messages → trim to latest 10 messages → SET with 4-hour expiration.
- Redis is the primary short-lived session store; PostgreSQL should not be used as the high-frequency chat cache.
- Failure policy: if Redis is unavailable, fail gracefully rather than inventing missing conversation context; the exact fallback policy should be agreed during implementation review.

10. Topic Classification

- Required categories: jargon, product, market, personal advice.
- Jargon: educational terminology and concepts.
- Product: questions about financial products or product concepts.
- Market: general market/economic context questions.
- Personal advice: requests for individualized buy/sell/investment decisions or personalized recommendations.
- Start with a lightweight deterministic classifier for the foundation. Keep a classifier interface so it can later be replaced by an ML/LLM classifier without changing the API.

11. Guardrails and SEBI Deflection

- Personal-advice requests must be intercepted before normal LLM generation.
- Required deflection: "For personalised advice, please consult a SEBI-registered advisor."

- The guardrail should be deterministic at the decision boundary. LLM prompting is a secondary safety mechanism, not the sole enforcement mechanism.
- Examples of requests that should be deflected include individualized requests such as which stock a user should buy or sell or where a specific amount should be invested.
- Normal educational questions should continue through the grounded RAG/LLM flow.

12. RAG and Prompt Architecture

- Task 1 establishes the interface for RAG; Task 2 implements question → embedding → top-5 retrieval → prompt → LLM → response.
- Required system prompt: "You are a friendly financial learning assistant for PaiseWise. Answer only using the provided context. If you don't know, say so. Never give specific investment advice."
- The prompt should separate trusted system policy from retrieved knowledge and user conversation context.
- The assistant must not fill gaps with unsupported financial claims when retrieved context is insufficient.

13. Streaming Architecture

- Use FastAPI StreamingResponse with an asynchronous generator.
- Flow: LLM output chunk/token → async generator → StreamingResponse → client.
- Streaming reduces perceived wait time and enables the mobile typewriter effect required by the task.
- Conversation persistence should happen after the complete assistant response has been assembled, with care taken to handle interrupted streams.

14. Feedback Design

- Endpoint: POST /ai/chat/feedback.
- Payload: responseId + rating.
- Supported ratings: up or down.
- Persist feedback with response ID, user ID, session ID where available, category and timestamp.
- Task 3 will use these records for a weekly thumbs-up/down ratio per question category.

15. API Contracts

```
POST /ai/chat
Request:
{
  "userId": "user_123",
  "sessionId": "session_456",
  "message": "What is diversification?"
}
Response:
Streaming text / streamed chunks.

POST /ai/chat/feedback
Request:
{
  "responseId": "response_123",
  "rating": "up"
}
Response:
{
  "status": "success"
}
```

- Optional operational endpoint: GET /health for service verification.

16. Conceptual Data Model

- users: id, created_at.
- user_profiles: user_id, goal, learning_level, kyc_status, updated_at.
- portfolio_summaries: user_id, summary/holding summary, updated_at.
- chat_responses: id, user_id, session_id, question, response, category, created_at.
- chat_feedback: id, response_id, user_id, rating, created_at.

- Redis remains separate from this durable relational model.

17. Project Structure

```
paise_wise_ai/
├── app/
│   ├── main.py
│   ├── api/routes/
│   │   ├── chat.py
│   │   ├── feedback.py
│   ├── core/
│   │   ├── config.py
│   ├── models/
│   ├── schemas/
│   ├── services/
│   │   ├── chat_service.py
│   │   ├── guardrail_service.py
│   │   ├── classifier_service.py
│   │   ├── context_service.py
│   │   ├── rag_service.py
│   │   ├── llm_service.py
│   │   ├── response_validator.py
│   ├── repositories/
│   │   ├── redis_repository.py
│   │   ├── feedback_repository.py
│   ├── tests/
│   ├── docs/
│   ├── .env
│   ├── .env.example
│   ├── requirements.txt
│   └── README.md
```

- This is a modular-monolith structure: simple to develop locally, but with clean service boundaries that can later be separated if scale requires it.

18. Security and Privacy

- Use HTTPS in deployed environments.
- Authenticate users before reading profile or portfolio context.
- Authorize access to the session identified by userId/sessionId.
- Keep credentials and API keys outside source control.
- Validate message length and payload shape.
- Do not log secrets or unnecessary portfolio/conversation content.
- Apply rate limiting at the gateway/API layer in production.
- Treat guardrails and output validation as mandatory controls for a financial-learning product.

19. Reliability and Error Handling

- 400: invalid request payload.
- 401: authentication failure.
- 403: unauthorized resource/session access.
- 429: rate limit exceeded.
- 503: unavailable AI/infrastructure dependency.
- 500: unexpected internal failure.
- Return controlled error messages; never expose stack traces or secrets.
- LLM/vector-store failures must not result in fabricated financial information.

20. Observability

- Capture request ID, user/session identifiers in privacy-safe form, question category, retrieval latency, model latency, total latency, response ID and error type.
- Use structured logs so production incidents can be traced across API, Redis, database and model services.
- Avoid logging full sensitive conversations by default.

21. Testing Strategy

- Unit tests: topic classification, guardrail decisions, context construction, message-window trimming.
- Integration tests: FastAPI + Redis + PostgreSQL interactions.
- API tests: valid/invalid chat requests and feedback requests.
- Safety tests: personal-advice questions must return the SEBI deflection.
- Conversation tests: new session, existing session, more than 10 messages and expired session.
- Streaming tests: verify the endpoint returns a streaming response and handles interrupted generation safely.
- Task 3 performance requirement: 100 concurrent chat sessions and response time below 8 seconds per message.

22. Task 1 Acceptance Criteria

- Technical architecture is documented and reviewable.
- POST /ai/chat contract is defined.
- Last 10 messages + user profile + portfolio summary are defined as the conversation context.
- Redis key and 4-hour TTL are defined.
- Guardrail categories and SEBI deflection are defined.
- StreamingResponse architecture is defined.
- Feedback endpoint and database persistence are defined.
- Security, error handling and observability are defined.
- Project structure and testing strategy are defined.
- Document is ready for team review.

23. Implementation Plan

- Step 1: create the FastAPI project skeleton.
- Step 2: configure PostgreSQL and Redis.
- Step 3: implement schemas and repositories.
- Step 4: implement topic classification and deterministic guardrails.
- Step 5: implement conversation context and 10-message Redis window.
- Step 6: implement the LLM interface with a mock provider for local testing.
- Step 7: implement StreamingResponse.
- Step 8: implement feedback persistence.
- Step 9: implement RAG and the real LLM pipeline as Task 2.
- Step 10: add response validation, analytics and performance testing as Task 3.

24. Architecture Decision Record

- Decision: modular monolith for the initial implementation.
- Reason: Task 1 needs a clear, testable design and the user can develop it locally without introducing unnecessary distributed infrastructure.
- Decision: Redis for session state and PostgreSQL for durable records.
- Reason: Redis is appropriate for fast, expiring conversation state; PostgreSQL provides durable relational storage.
- Decision: deterministic guardrail before LLM generation.
- Reason: safety decisions should not rely only on probabilistic model behavior.
- Decision: provider-neutral LLM/RAG interfaces.
- Reason: allows model/provider changes without rewriting the API and orchestration layers.

25. Scope Boundary

- Task 1 is the design and foundation.
- Task 2 implements the RAG pipeline, required system prompt, streaming response, Redis storage and topic detection.
- Task 3 implements deeper multi-turn context, user-profile injection, response validation, feedback collection/analytics and the 100-concurrent-session performance test.
- Airflow/Kafka requirements belong to later portfolio/churn tasks; MLflow/model registry and shadow deployment belong to Task 7.
- Therefore, Task 1 should not be overloaded with later-task infrastructure unless required for a local integration test.